CPE 462
VHDL: Simulation and Synthesis

Exam Comments

Comment #1

- Not a good idea to initialize an internal signal with a varying input signal.

- May compile/simulate, but it will lead to warnings and trouble in hardware.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity question1 IS
    port(
     a: in std_logic_vector(3 downto 0);
     s: in std_logic_vector(2 downto 0);
     f: out std_logic_vector(3 downto 0)
);
end entity;

architecture arch of question1 is
SignalINC:bit_vector(3 downto 0):=to_BitVector(a);
begin

(... some code here ...)
end
```

Comment #2

- May compile but its a bad idea to use a use a single bit in a bus as a selector and all bits inside the bus for else statements.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity question1 IS
    port(
     a: in std_logic_vector(3 downto 0);
     s: in std_logic_vector(2 downto 0);
     f: out std_logic_vector(3 downto 0)
);
end entity;

architecture arch of question1 is
(... some code here ...)
begin
   (... some code here ...)

   f<=MUXh when s(2)='1'
      else Reg2 WHEN s ="111"
      else to_StdLogicVector(MUXl);

   (... some code here ...)

end
```

Comment #3

- Specifying a the entire contents for a generic bus can't be done like this.
- We are specifying the contents of a '4' bit bus... and not 'n' bits.
- Use *(others=>'Z') when others*

```
library  ieee;
use ieee.std_logic_1164.all;

entity exam1_2 is
    generic  (n: integer := 5);
    port (A: in std_logic_vector (n-1 downto 0);
    S : in std_logic_vector (2 downto 0);
    F : out std_logic_vector (n-1 downto 0)
    );
end exam1_2;

architecture dataflow of exam1_2 is
(... some code here ...)
begin
(... some code here ...)

    with S(1 downto 0) select
    increment <= A+1 when "00",
    A-1 when "01",
    A+2 when "10",
    "ZZZZ" when others;

(... some code here ...)
end dataflow;
```

Comment #4

- Here is a nice way to perform a shift using std_logic_vectors.

```
library ieee;
use ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all;

entity question1 is
    port (a : in  std_logic_vector(3 downto 0);
          s : in  std_logic_vector (2 downto 0);
          f : out std_logic_vector (3 downto 0)
    );
end;

architecture number1 of question1 is
  signal arith, logic: std_logic_vector(3 downto 0);
Begin
    with s(1 downto 0) select
     arith <=
        to_stdlogicvector(to_bitvector(a) srl 1) when "00",
        to_stdlogicvector(to_bitvector(a) sll 2) when "01",
        a when "10",
        NOT a when "11",
        "ZZZZ" when others;

     (... some code here ...)
end architecture;
```

Comment #5

- In generic code you can't have signals with hardwired sizes unless they really don't change!

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity question1 is
generic (n: integer := 5);
port (a : in  std_logic_vector(n-2 downto 0);
    s : in  std_logic_vector (2 downto 0);
    f : out std_logic_vector (n-2 downto 0));
end entity;

architecture dataflow of question1 is
signal shifter: std_logic_vector  (3 downto 0);
signal incrementer: std_logic_vector (3 downto 0);
signal x:bit_vector(3 downto 0);
begin
     (... some code here ...)
end architecture;
```

Comment #6

- In a generic test-bench the port names for the block you are testing **must** be the same and they **must** have the same generic size too.

```
library ieee;
use ieee.std_logic_1164.all;

entity test_question1 is
end;

architecture bench of test_question1 is
constant n:positive:=7;
component question1
    port (a : in  std_logic_vector(3 downto 0);
          s : in  std_logic_vector (2 downto 0);
          f : out std_logic_vector (3 downto 0));
  end component;

  signal a : std_logic_vector (3 downto 0);
  signal s : std_logic_vector (2 downto 0);
  signal f : std_logic_vector (3 downto 0);
  signal shifter: std_logic_vector(3 downto 0);
  signal incrementer: std_logic_vector (3 downto 0);

     (... some code here ...)
```

Comment #7

- Here is a pretty good way to do a generic bit inversion.

```
(... some code here ...)

entity question1 is
    generic (n: integer :=5);
    port(a:in std_logic_vector(n-1 downto 0);
    s:in std_logic_vector (2 downto 0);
    f:out std_logic_vector(n-1 downto 0));
end question1;

architecture question1 of question1 is
(... some code here ...)
begin
    temp1<=to_bitvector(a);
         x: for i in a'range generate
        temp2(i)<=a(i);
    end generate;

    with s(1 downto 0) select
    shift<= to_stdlogicvector(temp1 srl 1) when "00",
    to_stdlogicvector(temp1 sll 2) when "01",
    a when "10",
    temp2 when OTHERS;

(... some code here ...)
```

Comment #8

- Here is a clever way to do a non-generic bit inversion

- This has to be the best answer on question #1

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

entity question1 is
    port (a : in  std_logic_vector(3 downto 0);
          s : in  std_logic_vector (2 downto 0);
          f : out std_logic_vector (3 downto 0)
    );
end question1;

architecture myarch of question1 is
begin
f<= to_stdlogicvector(to_bitvector(a) srl 1) when s="000" else
a when s="010" else
to_stdlogicvector(to_bitvector(a) sll 2) when s="001" else
a(0)&a(1)&a(2)&a(3) when s="011" else
a+1 when s="100" else
a-1 when s="101" else
a+2 when s="110" else
"ZZZZ";
end myarch;
```

Comment #9

- This will not work. The use of arithmetic operators to perform logic operations is very tricky with vectors

- We should stay away from logic operations anyway!

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

Entity question1 IS
    PORT (a: IN STD_LOGIC_vector(3 downto 0);
    sel:in STD_LOGIC_VECTOR(2 downto 0);
    mux:OUT STD_LOGIC_vector(3 downto 0));
END question1;
architecture question1 of question1 is
signal shift:STD_logic_vector(3 downto 0);
signal inc :Std_logic_vector(3 downto 0);

begin
(... some code here ...)
mux<=  ((shift) * sel(2))+(inc * not sel(2));
end question1;
```

Comment #10

- The size of S ought to remain the same.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.std_logic_arith.all;

entity test_question2 is
    generic (n: integer := 4);
    port (a: IN bit_vector (n downto 0);
    s: IN std_logic_vector (n/2 downto 0);
    f: OUT std_logic_vector (n downto 0));
end entity;

architecture myarch of test_question2 is
signal shifter, incrementer: std_logic_vector (3 downto
0);
begin
```